

HLX: A Unified Pipelined Architecture for Optimized Performance of Hybrid Transformer-Mamba Language Models

In-Jun Jung
KAIST
Daejeon, Republic of Korea
injun@kaist.ac.kr

Jaeha Min
KAIST
Daejeon, Republic of Korea
derekmin0807@kaist.ac.kr

Gyeongrok Yang
KAIST
Daejeon, Republic of Korea
toddlerf@kaist.ac.kr

Joo-Young Kim
KAIST
Daejeon, Republic of Korea
jooyoung1203@kaist.ac.kr

Abstract

The rapid increase in demand for long-context language models has revealed fundamental performance limitations in conventional Transformer architectures, particularly their quadratic computational complexity. Hybrid Transformer-Mamba models, which interleave attention layers with efficient state-space model layers such as Mamba-2, have emerged as promising solutions combining the strengths of both Transformer and Mamba. However, maintaining a high compute utilization and performance across workloads (e.g., varying sequence length and batch size) in the Hybrid models is challenging due to their heterogeneous compute patterns and shifting performance bottlenecks between the two key computational kernels: FlashAttention-2 (FA-2) and State-Space Duality (SSD).

In this paper, we introduce HLX, a unified pipelined architecture designed to ensure optimized performance across workloads for Hybrid models. Through detailed kernel-level analysis, we identify two key blockers that limit compute utilization: inter-operation dependencies in FA-2 and excessive memory traffic in SSD. To overcome these hurdles, we propose two novel fine-grained pipelined dataflows named PipeFlash and PipeSSD. PipeFlash effectively hides operational dependencies in attention computations, while PipeSSD firstly introduces the fused pipelined execution for SSD computations, substantially enhancing data reuse and reducing memory traffic. In addition, we propose a unified hardware architecture that can process both PipeFlash and PipeSSD in an efficient pipelining scheme to maximize the compute utilization. Finally, across sequence lengths from 1K to 128K, the proposed HLX architecture achieves up to 97.5% and 78.4% compute utilization for FA-2 and SSD, respectively, resulting in an average speedup of 1.75 $\times$ and 2.91 $\times$ over A100, and an average 2.78 $\times$ (FA-2), 1.84 $\times$ (FA-3), and 4.95 $\times$ speedups over H100. For end-to-end latency and batching, HLX achieves a 1.56 $\times$ and 1.38 $\times$ speedup over A100 and a 2.08 $\times$ and 1.76 $\times$ (1.84 $\times$ and 1.72 $\times$) speedup when running FA-2 (FA-3) on H100. It also significantly reduces area and power consumption by up to 89.8% and 63.8% compared to GPU baselines.

CCS Concepts

• **Computer systems organization** → **Neural networks**; • **Hardware** → **Hardware accelerators**.

Keywords

Hybrid Transformer-Mamba Language Model, FlashAttention-2, Mamba-2, State-Space Duality, Fusion, Dataflow, Accelerator

ACM Reference Format:

In-Jun Jung, Gyeongrok Yang, Jaeha Min, and Joo-Young Kim. 2025. HLX: A Unified Pipelined Architecture for Optimized Performance of Hybrid Transformer-Mamba Language Models. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3725843.3756115>

1 Introduction

The Hybrid Transformer-Mamba language models [11, 15, 16, 27, 45, 49, 51] are emerging as a promising solution to overcome critical limitations of conventional attention-based large language models (LLMs). By combining Transformers and state-space models (SSMs), such as Mamba [11, 19], these Hybrid models mutually compensate for the drawbacks of each architecture, thereby achieving both accuracy and efficiency. Specifically, Mamba, with its linear computational complexity and constant memory usage, stems from the SSM, mitigates Transformer’s quadratic computational complexity and increased memory usage due to its key-value (KV) cache, thereby enabling the processing of longer sequences. Meanwhile, the strong language modeling capabilities of the attention compensate for the degraded recall and in-context learning capabilities of Mamba. As a concrete example, the recent Hybrid model [49] achieves 2.5 times faster inference compared to Mistral [20], Llama-3.1 8B [18], and Mixtral 8 $\times$ 7B [21] with 8 times less KV cache memory at a sequence length of 256K, along with improved accuracy.

The complementary advantages of Hybrid models directly stem from their two core computational kernels: attention and SSM. Consequently, maintaining consistent and efficient performance across varying sequence lengths critically depends on effectively managing these two computational kernels. As illustrated in Fig. 1, for sequence lengths below 128K, the overall latency is dominated by the numerous Mamba-2 layers despite their linear complexity. In contrast, as the sequence length increases beyond 128K, the



This work is licensed under a Creative Commons Attribution 4.0 International License. *MICRO '25, Seoul, Republic of Korea*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1573-0/25/10
<https://doi.org/10.1145/3725843.3756115>

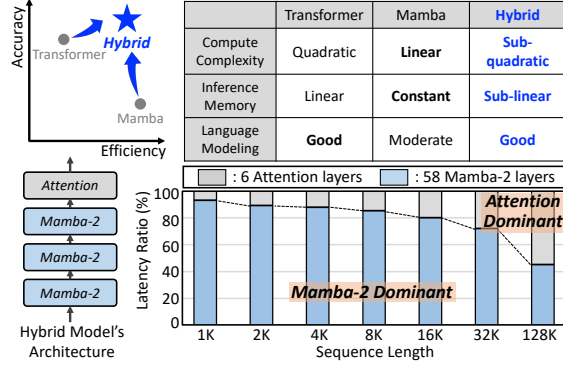


Figure 1: The comparison between Transformer, Mamba, and Hybrid models. The latency breakdown of the Hybrid-2.7B model on an A100 GPU according to the sequence length.

quadratic complexity of attention layers starts to dominate the latency, highlighting a shift in the performance bottleneck.

Although various software (SW) approaches have been proposed to optimize two key computations on conventional hardware (HW), such as GPUs, both Transformer’s attention and Mamba-2 still suffer from suboptimal performance. For attention layers, FlashAttention-2 (FA-2) [10] significantly reduced memory access by merging softmax and matrix multiplication (MatMul) into a single fused kernel while applying tiling and recomputation techniques to compute results directly without storing intermediate data. In addition, FA-2 parallelizes along the sequence length dimension to maximize compute utilization. However, FA-2 still suffers from a dependency between MatMul and non-MatMul operations related to softmax within the attention mechanism. This limitation arises from FA-2’s synchronous execution, which restricts the ability to overlap non-MatMul computations with MatMul, thereby inhibiting effective latency hiding [47]. Consequently, as the sequence length increases, compute utilization is saturated at about 61% and 49% on A100 [34] and H100 [36], respectively. To address this low compute utilization, FlashAttention-3 (FA-3) [47] was recently introduced, building upon the Hopper GPU architecture, such as H100. The H100 supports asynchronous execution by incorporating a specialized Tensor Memory Accelerator (TMA) and SW pipelining with warp-specialization [5], allowing FA-3 to overlap non-MatMul and MatMul operations. Nevertheless, despite this asynchronous execution, the compute utilization of FA-3 on the H100 also saturates at around 61%.

For Mamba-2 layers, a state-space duality (SSD) algorithm is introduced to overcome the limited parallelism of SSM, the core computation of Mamba-1, and enhance inference speed [11]. SSD leverages the fact that SSMs are semiseparable matrices. It decomposes these matrices into tiles to increase the number of MatMul operations and improve parallelism rather than computing the entire semiseparable matrices recurrently. MatMul can be viewed as SSM in a tiled form, where input and output sequences are partitioned. However, SSD remains memory-bound, with compute utilization saturating at only about 26.9%. This is because SSD has a higher number of memory-intensive element-wise operations compared

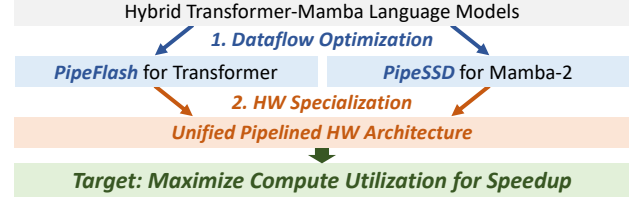


Figure 2: Overview of HLX.

to the attention mechanism, and the high volume of intermediate data generated during the computations, combined with a lack of data reuse, results in relatively high DRAM traffic. Additionally, SSD’s recurrent nature prevents it from achieving parallelism along the sequence length dimension like FA-2, resulting in low compute utilization. This memory-bound operation can be sped up through kernel fusion, but to the best of our knowledge, no research has yet fused SSD.

Motivated by these challenges, in this paper, we propose **HLX**, the first unified Hybrid Transformer-Mamba Language model Xcelerator providing optimized performance on varying sequence lengths and batch sizes. HLX supports the newly proposed fine-grained pipelined dataflows specifically tailored to enhance FA-2 and SSD computations. For attention layers (FA-2), HLX introduces PipeFlash, a refined, fine-grained pipelined dataflow that addresses the issue of dependencies preventing latency reduction, thereby substantially enhancing compute utilization. For the Mamba-2 layers (SSD), we propose PipeSSD, a novel fused SSD algorithm that employs a fine-grained pipelined execution to fuse block-based SSD computations. HLX natively supports both fine-grained pipelined dataflows in the unified HW. This approach significantly reduces the DRAM traffic and volume of intermediate data while effectively lowering on-chip memory requirements and enhancing compute utilization. Existing GPU architectures inherently struggle to support HLX’s proposed PipeFlash and PipeSSD efficiently. First, although the proposed fused SSD algorithm (i.e., without a fine-grained pipelining) effectively increases the data reuse, it still generates more than twice the amount of intermediate data compared to FA-2, exceeding the per streaming multiprocessor (SM) memory capacity on modern GPUs like A100 and H100 and causing register spilling and reduced occupancy. FA-3 also suffers from register pressure, resulting in a limited block size [47]. Second, although a fine-grained pipelined dataflow could be an attractive solution to mitigate these issues by reducing non-MatMul computation overhead and register spilling, current GPUs exhibit structural limitations in supporting it efficiently. Warp-specialized pipelines are heterogeneous, requiring different resources at each stage, yet the SM—based on a SIMT execution model that assumes uniform warp execution—cannot efficiently handle such pipeline parallelism [9, 33, 43]. Furthermore, while the TMA efficiently supports coarse-grained memory tile transfers, overhead persists for fine-grained memory access [9].

Consequently, we need a specialized HW architecture to achieve the maximum performance from PipeFlash and PipeSSD, as illustrated in Fig. 2. Our proposed HLX achieves high compute utilization by seamlessly handling FA-2 and fused SSD computations,

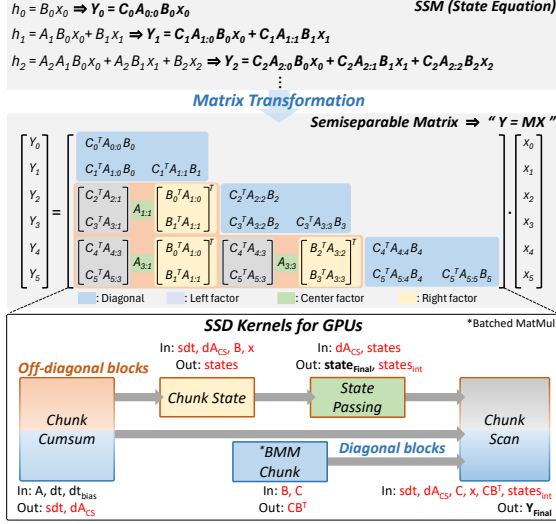


Figure 5: The block decomposition method of the SSD algorithm is illustrated. It consists of five kernels for GPUs. There is a possibility of reusing the intermediate data.

temporal dynamics of the model. These parameters (A , dt , x , B , C) interact within the SSM to compute the state equations (see Fig. 3). After the SSM computation, the output Y undergoes z-gating, followed by RMSNorm, an output linear projection, and a residual connection. Through these computations, Mamba-2 demonstrates superior computing and memory efficiency over Transformers, achieving enhanced performance for modeling long sequences.

2.3 Hybrid Transformer-Mamba Models

While Mamba-2 successfully overcomes the efficiency limitations of Transformers when modeling long sequences and outperforms them across various natural language processing (NLP) tasks, it still lags behind Transformers in tasks such as in-context learning and recall [4, 51]. This is primarily due to Mamba-2 selectively compressing and storing input tokens within a fixed-size state, leading to gradual information decay over time.

To address these limitations, Hybrid Transformer-Mamba models [11, 15, 16, 27, 45, 49, 51] have recently been proposed. By synergistically leveraging the complementary strengths of both architectures, the Hybrid model not only demonstrates superior performance compared to traditional Transformer-based models but also supports significantly longer sequence lengths. Fig. 3 also shows the architecture of the Hybrid model. Specifically, this model sequentially interleaves the Transformer's attention layers and Mamba-2 layers according to a specific ratio.

2.4 FlashAttention-2 and -3 (FA-2 and FA-3)

FA-2 [10] is an optimized solution designed to alleviate the memory bandwidth bottleneck found in conventional attention mechanisms, resulting in an inference speedup. It divides the attention computation into small blocks and fuses key operations (e.g., MatMul,

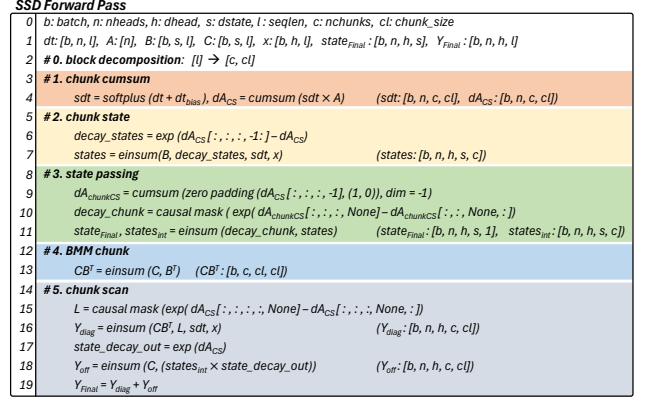


Figure 6: Process of SSD algorithm.

softmax, masking, etc.) within a single kernel. This approach recomputes intermediate results, such as score and probability matrices, on the fly without storing them in external DRAM, significantly reducing DRAM traffic. Moreover, there is no dependency between query (Q) blocks, enabling parallel processing along the sequence length dimension while maintaining head and batch dimension parallelism, as shown in Fig. 4. Each block performs its operations in the order of QK^T , *local softmax*, *PV*, and *update output* (O). FA-3 [47] extends FA-2 by redesigning the kernel to leverage new Hopper-specific capabilities such as asynchrony and low-precision. It employs a warp specialization that creates separate "producer" warps for data movement and "consumer" warps for computation, allowing these tasks to be overlapped. It further improves efficiency by interleaving the slower softmax with the faster MatMul operations. Additionally, FA-3 adds support for FP8 precision to nearly double computational throughput, while mitigating accuracy loss.

2.5 State-Space Duality (SSD)

SSD is a hardware-efficient algorithm for the parallel processing of input sequences (SSM computations in Fig. 3 are executed in parallel along the sequence length dimension), introduced in Mamba-2. It overcomes the limitation of Mamba-1, where the recurrent nature of SSM computations lacked the necessary MatMuls, preventing full utilization of modern hardware such as GPUs and TPUs [24]. As illustrated in Fig. 5, SSD leverages the insight that SSM operations can be represented as semiseparable matrices, enabling efficient linear (recurrent) computations as well as hardware-friendly quadratic (attention-like) operations. Rather than performing recurrently over the entire semiseparable matrix at once, SSD employs a block decomposition strategy. Specifically, the diagonal blocks compute independent local SSM outputs within each block of the input sequence. These computations are efficiently implemented using MatMuls and can be performed entirely in parallel. Off-diagonal computations are factorized into three components—right, center, and left factors—to efficiently propagate state information between blocks. The right factors summarize how input data within a block propagates forward by computing each block's states. The center factors then efficiently combine these states across blocks through

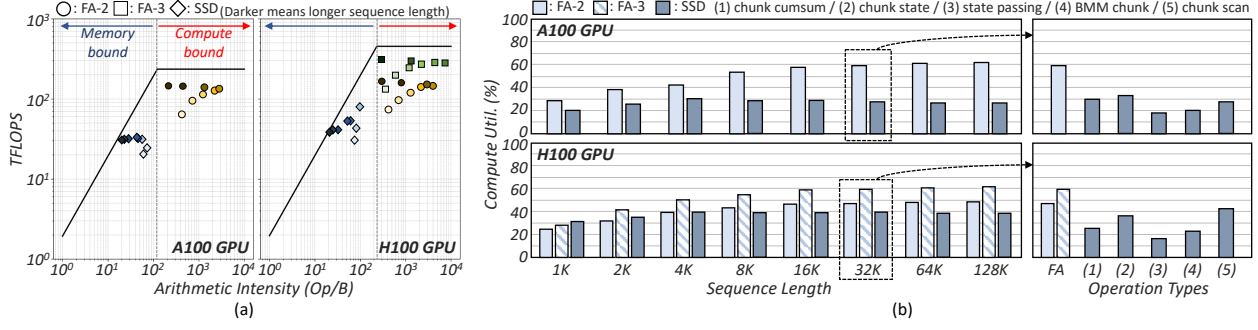


Figure 7: (a) The roofline graph and (b) compute utilization for FA-2 and SSD on GPU with varying sequence lengths.

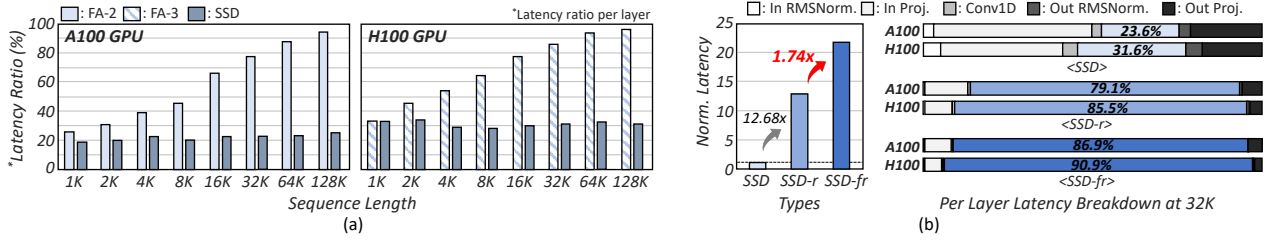


Figure 8: (a) Latency breakdown of (a) FA-2 and (b) SSD on GPU.

cumulative multiplications (1-semiseparable multiplications), effectively propagating global state information forward in time. Finally, the left factors project these accumulated global states into outputs for each block, correctly integrating prior context.

To efficiently process these operations on GPUs, SSD is composed of five kernels: chunk cumsum, chunk state, state passing, batched MatMul (BMM) chunk, and chunk scan. Each kernel is implemented to handle both diagonal and off-diagonal blocks in a highly parallel manner. Fig. 6 shows the process of SSD kernels. The chunk cumsum kernel computes sdt and dA_{CS} parameters that reflect the decay of state and input data over time, while the chunk state kernel uses these parameters along with B and x to compute the right factor representing the states. The states computed within each block are then updated by the influence of previous input tokens through the state passing process, producing the final state ($state_{Final}$) as the center factor. For diagonal block operations, the BMM chunk kernel performs a MatMul between B and C . Finally, the chunk scan kernel calculates both the left factor (yielding the off-diagonal output, Y_{Off}) and the diagonal output (Y_{Diag}), which are combined to produce the final output (Y_{Final}).

3 Computation Analysis

3.1 Arithmetic Intensity & Compute Utilization

FA-2 and FA-3 employ a fusion technique to reduce DRAM accesses and increase data reuse, thereby achieving a relatively high arithmetic intensity (Op/B) (see Fig. 7(a)). Consequently, as the sequence length increases, the Op/B continues to rise. However, when the sequence length exceeds 16K (32K for FA-3), the increase in the KV cache leads to a reduction in the Op/B, although it still

remains compute-bound. In terms of compute utilization, there is a gradual increase with longer sequence lengths, but it saturates at approximately 61% at 128K, as shown in Fig. 7(b). This is because FA-2 adheres to a synchronous nature, limiting the overlap of non-MatMul with MatMul computations. For FA-3, although it supports asynchronous 2-stage pipelining for hiding non-MatMul computations, it still suffers from pipeline-agnostic HW and register pressure.

In contrast, SSD remains memory-bound with a very low Op/B, as shown in Fig. 7(a). Even when the sequence length is increased up to 128K, the Op/B hardly increases, and the compute utilization remains at about 27% on A100 and 38% on H100, which is roughly 2.3 $\times$ and 1.6 $\times$ lower than that of FA-2 and FA-3, respectively (see Fig. 7(b)). Although SSD operations minimize recurrent operations and maximize MatMul operations to enhance parallelism, they still remain sub-optimal on the GPUs. This is because SSD itself has a higher proportion of memory-intensive operations, such as element-wise operations, compared to FA-2, and it involves many Einsum operations between multi-dimensional tensors. Additionally, a significant amount of intermediate data is generated during each operation, and this data is not immediately reused. To maximize these data reuse opportunities and shift the workload into the compute-bound, a block-level fusion technique similar to that of FA-2 is essential for SSD operations.

3.2 Latency Breakdown

Owing to their quadratic computational complexity, FA-2 and FA-3 dominate the attention layer latency once the sequence length exceeds 16K, as shown in Fig. 8(a). Therefore, accelerating FAs is

crucial for speeding up the attention layer of the Hybrid model. In contrast, because SSD has a linear characteristic, even as the sequence length increases, it maintains a nearly constant proportion within the Mamba-2 layer, accounting for an average of about 24% on the A100 and 32% on the H100 for sequence lengths ranging from 1K to 128K at the given Hybrid model size. This makes it the second-largest component in the Mamba-2 layer. Since the number of Mamba-2 layers is dominant in the Hybrid model, accelerating this part is also an essential factor.

However, SSD remains memory-bound, so acceleration through fusion techniques is essential. When implementing operations with a large amount of intermediate data, such as SSD, as a fused single kernel, the required shared memory for each operation becomes a limiting factor [25]. Moreover, if the shared memory requirement exceeds the GPU's available capacity, the kernel will not be able to execute [10, 47]. Consequently, a latency breakdown is performed on both the PyTorch [44]-based reference SSD (SSD-r) that performs the same operations as the SSD presented in [11], and the fused reference SSD (SSD-fr) implemented based on SSD-r using the proposed fused SSD algorithm (described in Section 4). The results showed that the latency of SSD-fr increased by $1.74\times$ compared to SSD-r, as shown in Fig. 8(b).

This outcome is mainly due to the fact that, despite the proposed fused SSD algorithm maximizing data reuse and efficiently reducing DRAM traffic, it still involves over twice as much intermediate data as FA-2. For instance, fused SSD requires 642KB of on-chip memory per block, whereas FA-2 requires only 321KB. This exceeds the per-SM memory capacity of both A100 (256KB of register files and up to 164KB of shared memory) and H100 (256KB of register files and 224KB of shared memory) [34–37], causing register pressure. Since each thread is allocated a finite number of registers, exceeding this limit causes register spilling, where excess data is stored in local memory or, even worse, global memory. This not only increases memory access latency but also reduces the number of threads that can be concurrently scheduled (i.e., lower occupancy), ultimately degrading parallel performance. In addition, fused SSD suffers from sequential dependencies introduced by fusion. In particular, unlike FA-2, SSD exhibits not only row-wise dependency but also column-wise dependency due to its recursive characteristics, which results in a loss of parallelism along the sequence length dimension. Therefore, the fused SSD must also reduce the amount of intermediate data generated through fine-grained pipelining to decrease the required on-chip memory capacity and mitigate sequential dependencies.

3.3 Motivation for Specialized Architecture

A new HW architecture is necessary to support both FA-2 and SSD efficiently and maximize their performance. Even advanced methods like FA-3, which utilize block-level asynchrony, still suffer from register pressure (2-stage pipeline doubles the amount of intermediate data for FA-2) and saturated compute utilization, failing to completely hide non-MatMul latencies. Furthermore, so far, FA-3's complex optimization results in a highly specialized implementation whose performance is primarily optimized for specific head dimensions (e.g., 64, 128, 256), potentially leading to performance degradation for other sizes. For SSD, the unfused version

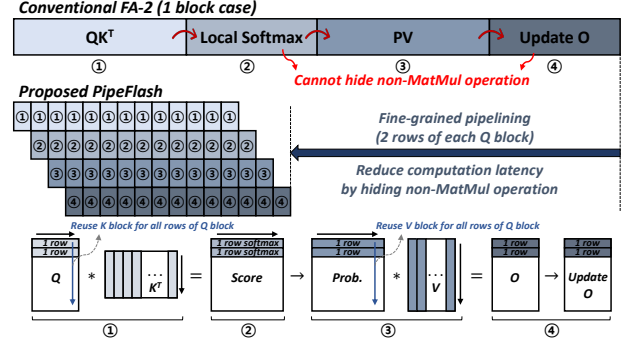


Figure 9: Proposed PipeFlash dataflow.

has critically low compute utilization due to its low Op/B, while the fused SSD, on the other hand, suffers from limited register file and shared memory capacity on GPUs, making it challenging to implement as a single fused kernel. Even when implemented, it often results in performance degradation. Moreover, fused SSD introduces sequential dependencies due to its recurrent nature, which further limits parallelism. To overcome these challenges, a fine-grained pipelined dataflow is an attractive solution, as it can mitigate both non-MatMul overheads and register spilling. However, current GPUs cannot efficiently support such fine-grained pipelining due to fundamental architectural constraints. This is because each pipeline stage is a warp with a unique program and different resource requirements (e.g., registers, functional units). This heterogeneity conflicts with the GPU's SIMT execution model, which assumes uniform execution and thus incurs significant scheduling overhead and resource contention for these workloads [9, 29, 33, 43]. In addition, advanced features like H100's TMA are designed for coarse-grained tile movements. This leaves a significant performance opportunity untapped, as many applications that could benefit from pipelining are characterized by fine-grained streaming and gather memory access patterns that are not well supported by existing hardware [9].

4 Proposed HLX Architecture

4.1 Dataflow Strategy

PipeFlash. To speed up the conventional FA-2 by hiding the latency of non-MatMul operations, we propose PipeFlash. Unlike FA-2, which performs operations at the block-level, PipeFlash executes the attention mechanism's QK^T , *local softmax*, *PV*, and *update O* step-by-step through a pipeline at a finer granularity, processing two rows of *Q* at a time within each block. As shown in Fig. 9, this approach concurrently executes the *softmax* and *update O* operations with MatMul operations such as QK^T and *PV*, effectively hiding non-MatMul latency by mitigating the dependencies. Also, PipeFlash reduces the amount of intermediate data generated during the computation by $4.8\times$ compared to conventional FA-2. This reduction is attributed to the fine-grained pipelining, which reduces the size of the score and probability matrices generated during computation from 128KB to 1KB, respectively. Lastly, similar to FA-2, PipeFlash reuses *K* and *V* blocks by processing all rows in a

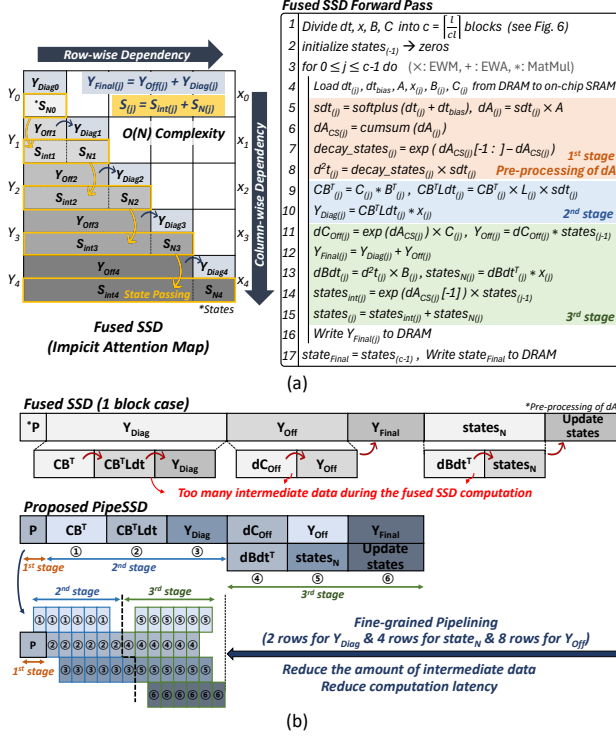


Figure 10: (a) Proposed fused SSD. By fusing block-level operations similar to FA-2, external DRAM accesses for intermediate data are reduced. (b) Proposed PipeSSD dataflow.

given Q block against them. However, PipeFlash uniquely performs this at a finer granularity to enable its pipeline.

PipeSSD. The proposed fused SSD performs block-level fusion and is composed of six operations: *pre-processing related to dA* (line 5-8), Y_{diag} (line 9-10), Y_{off} (line 11), Y_{final} (line 12), $states_N$ (line 13), and *update states* (line 14-15), as shown in Fig. 10(a). While FA-2 employs separate for loops for KV and Q , fused SSD uses a single for loop, achieving linear computational complexity with respect to sequence length. However, within each block, there is a column-wise dependency for transferring and updating the states (line 14) from the previous block to generate decayed states ($states_{int}$). In addition, a row-wise dependency arises from the addition between the transferred $states_{int}$ and newly generated $states_N$ to produce $states$ (line 15), as well as from combining the computed Y_{off} and Y_{diag} to produce Y_{final} (line 12). Moreover, to compute efficiently without external DRAM access, all the massive amounts of intermediate data generated during these processes must be stored on-chip.

Consequently, to efficiently process the fused SSD in a streaming manner and effectively reduce the on-chip memory requirement, PipeSSD, a fused SSD with fine-grained pipelining, is proposed. The PipeSSD divides the fused SSD into three stages, considering the dependencies between computations (see Fig. 10(b)). The computations related to Y_{off} and those related to $states_N$ can be operated simultaneously because they are independent computations. Thus, the PipeSSD combines those computations (3rd stage).

After completing the 1st stage for *pre-processing*, the operations for computing the 2nd stage (CB^T , $CB^T Ldt$, and Y_{diag}) are seamlessly executed through pipelining. Then, the concurrent pipelined execution of (dC_{off} and $dBdt$), (Y_{off} and $states_N$), and (Y_{final} and *update states*) are performed (3rd stage). The number of rows assigned to each operation is determined by considering the pipeline cycle. Through this process, PipeSSD reduces DRAM accesses by 6.8 $\times$ compared to conventional SSD and decreases the amount of intermediate data generated during computation by 11 $\times$, from 642KB to 58.5KB, while improving the compute utilization.

4.2 Hardware Architecture

Overall Architecture. As shown in Fig. 11(a), HLX consists of a top controller that manages the computation mode and loads (stores) data from (to) DRAM, a transpose unit, a unified reconfigurable streamlined core (URSC), and a global scratchpad (GS) connected to the URSC through a network-on-chip (NoC). The URSC comprises two dot-product engines (DPEs), a reconfigurable vector processing engine (RVPE), and an update engine (UpE). This HW architecture focuses on accelerating the sequential computations along with the sequence length dimension in both FA-2 and SSD while maintaining parallel processing along with the batch and head dimensions. To achieve this, the URSC serves as the core of HLX, and seamless pipelining between the DPE, RVPE, and UpE significantly enhances compute utilization and reduces computation latency.

Dot-Product Engine (DPE). Each DPE comprises 32 DPU lanes, with each DPU lane consisting of 8 DPUs. Each DPU is primarily designed for MatMul operations and is composed of 16 floating-point 16-bit (FP16) multipliers, an adder tree, and an accumulator (see Fig. 11(b)). Additionally, a demux is incorporated to output results without accumulation, enabling support for the conv1D operation within the Hybrid model. Within each DPU lane, while the DPUs share 16 broadcasted activations, each DPU receives distinct weights to perform row-wise MatMul operations for a fine-grained pipeline dataflow. After the computation, the 8 outputs produced by each DPU are forwarded—along with outputs from other DPU lanes—to either the RVPE or UpE for subsequent processing.

Reconfigurable Vector Processing Engine (RVPE). The RVPE consists of two reconfigurable vector processing units (RVPU) and a memory (VMEM) for storing intermediate data generated after pre-processing. Each RVPU, as shown in Fig. 11(c), comprises an addition/subtraction (add/sub) unit capable of processing 256 data elements, a unit that performs rowsum for FA-2 operations and cumulative-sum (cumsum) for SSD operations, two multiplication units, and a special function unit (SFU) for reciprocal, exponential (exp), max, log, square-root-mean (sqrt), and SiLU. It also includes a dedicated reconfigurable network that supports four operation modes among these units. Through this reconfigurable network (local NoC), the RVPU efficiently performs PipeFlash's *local softmax* (line 8 in Fig. 4), PipeSSD's *pre-processing* (line 5-8 in Fig. 10(a)), element-wise multiplication for computing Y_{diag} (line 9 in Fig. 10(a)), and element-wise multiplication for calculating $Y_{off}/states_N$ (line 11 and 13 in Fig. 10(a)).

Update Engine (UpE). The UpE consists of two Update Units (UpUs). Both PipeFlash and PipeSSD perform similar operations,

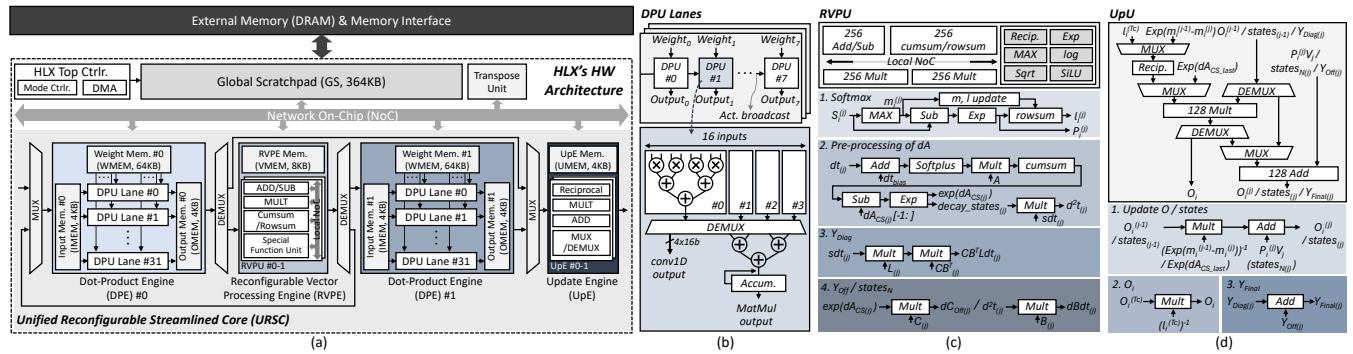


Figure 11: (a) Overall architecture of HLX. Microarchitecture of (b) dot-product unit (DPU) lanes, (c) reconfigurable vector processing unit (RVPU), and (d) update unit (UpU).

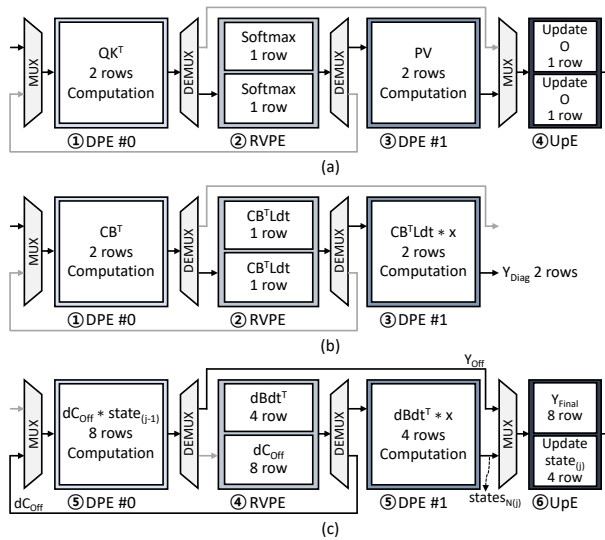


Figure 12: (a) PipeFlash mapping. (b) Mapping of Y_{Diag} computation and (c) that of Y_{Off} and states_N in PipeSSD.

namely *update O* and *update states*. In both operations, each element of the previous *O* and *states* is multiplied by a pre-computed exponentiated value, and the resulting product is then combined with the newly computed *PV* or *states_N* via element-wise addition to update the current block's *O* and *states* (line 10 in Fig. 4 and line 14-15 in Fig. 10(a)). As illustrated in Fig. 11(d), the UpU not only supports these update operations, but is also designed to compute the final *O* (*O_i*) after all iterations are complete (line 11 in Fig. 4), as well as to perform the element-wise addition between *Y_{Diag}* and *Y_{Off}* for *Y_{Final}* (line 12 in Fig. 10(a)). The final outputs produced by each UpU are stored in OMEM, and once the processing for each block is complete, the results are transferred to external DRAM.

Dataflow Mapping. The URSC is a key component of HLX’s architecture, enabling the computations of PipeFlash and PipeSSD to be efficiently mapped onto the URSC for a streamlined and effective operation. Fig. 12(a) illustrates how PipeFlash processes its computations within the URSC (see Fig. 9), while Figs. 12(b) and

(c) show how the 2nd and 3rd stages of PipeSSD (see Fig. 10(b)) are handled. In the case of PipeFlash, the operations for QK^T , *local softmax*, *PV*, and *update O* are sequentially executed in DPE#0, RVPE, DPE#1, and UpE, thereby hiding the *local softmax* and *update O* operations and achieving high compute utilization. PipeSSD’s Y_{Ddiag} operation proceeds in a manner very similar to that of PipeFlash. Specifically, the CB^T computed in DPE#0 is forwarded to the RVPE, where an element-wise multiplication produces $CB^T Ldt$. This result is then sent to DPE#1, where a MatMul operation computes the Y_{Ddiag} . The computed Y_{Ddiag} is temporarily stored in GS for subsequent Y_{Final} computation. After all row operations for Y_{Ddiag} are completed, the operations for Y_{Off} and $states_N$ follow. Unlike the Y_{Ddiag} operation that starts in DPE#0, each RVPU within the RVPE first processes the computations for $dBdt^T$ and $dCOff$, and then, via mux/demux, changes the data transmission direction by forwarding the results to DPE#0 and DPE#1, respectively. In DPE#0, a MatMul operation between the received $dCOff$ and the $states_{(j-1)}$ (previous states) computes Y_{Off} , while in DPE#1, a MatMul operation between the received $dBdt^T$ and x produces $states_N$. The computed Y_{Off} and $states_N$ are then forwarded to UpE, where Y_{Off} is added element-wise with the previously computed Y_{Ddiag} to produce the final result, Y_{Final} . Simultaneously, for updating states, an element-wise multiplication is performed between the $states_{(j-1)}$ and $exp(dACS[-1])$, which represents time-based decay. This result is then added element-wise with $states_N$ to obtain $state_{Final}$. In this way, compute utilization is enhanced to achieve a speedup while also reducing the amount of intermediate data generated during computation.

Pipeline Stage Balancing. To achieve pipeline stage balancing within the URSC, the design is based on its bottleneck—the MatMul operation in the DPE. Consequently, the RVPE and UpE engines, which have lighter workloads, are scaled to match the DPE’s throughput. The computation cycle of DPE is calculated as shown in Fig. 13: $\lceil d_{reduction}/DPU_{size} \rceil \times \lceil (d_{in} \times d_{out})/DPE_{size} \rceil$. Our architecture employs a simple yet powerful principle to achieve a balanced pipeline: controlling the number of rows processed by each engine. This principle is applied with flexibility depending on the operation. For instance, in PipeFlash, where QK^T and PV have the same computational workloads (FLOPs) with $block_{size}$

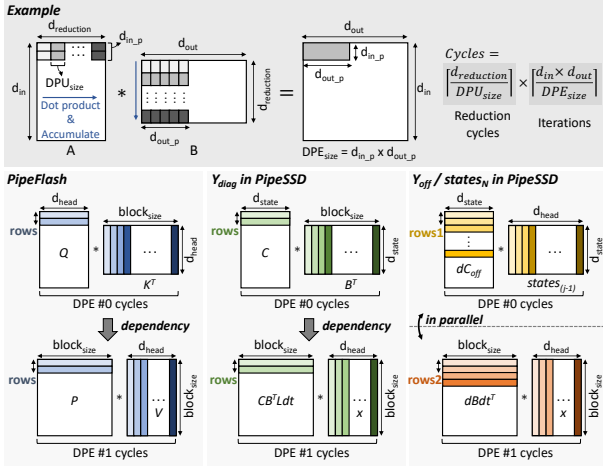


Figure 13: Pipeline Stage balancing of PipeFlash and PipeSSD.

and d_{head} , the number of rows processed is adjusted by a factor of $\lceil block_size / d_{head} \rceil$. Similarly, Y_{diag} stage of PipeSSD is balanced in the same manner as PipeFlash, although it introduces an additional dimension (d_{state}). Here, any mismatch with d_{head} , or $block_size$ not perfectly divisible by the d_{head} , can become minor sources of under-utilization. Conversely, $Y_{off}/states_N$ offer more flexible balancing by equalizing the total computation cycles rather than strictly matching the number of processed rows. This allows resource allocation tailored to the total FLOPs of each operation, ensuring that even with different numbers of processed rows, the final completion cycles align, thus preventing pipeline stalls. This architectural flexibility yields exceptional performance. When $block_size$, d_{head} , and d_{state} are all equal, the pipeline achieves nearly 100% compute utilization with single-row processing. Even when these dimensions are not identical, HLX maintains robust utilization by adjusting the number of processed rows to minimize inefficiency. As a result, HLX efficiently supports the decode phase without sacrificing compute efficiency, making it well-suited for auto-regressive inference, although in this paper, we focus only on accelerating the prefill phase of the Hybrid model. Notably, HLX also demonstrates great scalability. Across five configurations of the Mamba-2 model [11] (130M to 2.7B), compute utilization remains consistently high with less than 2% variation. This proves that HLX is a highly efficient and scalable architecture.

5 Evaluation

5.1 Methodology

Model and Accuracy. To evaluate HLX, we utilize the 2.7B Hybrid model (Mamba2attn-2.7B) from the GitHub repository [11], which provides the GPU-optimized FA-2 and SSD. This Hybrid model features a fundamental backbone structure that supports various models enhanced with modifications such as bigger model size [51], mixture-of-expert (MoE) [27, 49], and shared attention block [15, 16]. The attention layer of this model employs a multi-head attention operation consisting of 30 heads, and each head has a dimension of 128. In contrast, the SSD operation uses 80 heads, with

Table 1: HW Specifications for Comparison

HW Specifications of GPU and TPU			
	H100 GPU	A100 GPU	^{a)} TPU
Technology	4 nm	7 nm	16 nm
^{b)} Throughput	756 TFLOPS	312 TFLOPS	61.5 TFLOPS
Memory Bandwidth	2000 GB/s	1935 GB/s	450 GB/s
On-Chip SRAM Capacity	^{c)} 103.9 MB	^{c)} 84.3 MB	16 MB
DRAM Capacity	80 GB	80 GB	16 GB
Area	814 mm ²	826 mm ²	324 mm ²
Power Consumption	350 W	300 W	225 W
HLX Configuration for Comparison			
	HLX ⁶⁰ (Scaled to 7 nm)	HLX ³⁰ (Scaled to 7 nm)	HLX ⁶
Technology	14 nm	14 nm	14 nm
^{b)} Throughput	614.4 TFLOPS	307.2 TFLOPS	61.44 TFLOPS
On-Chip SRAM Capacity	30.4 MB	15.2 MB	3.04 MB
Area	475 mm ² (169 mm ²)	235.8 mm ² (83.9 mm ²)	47.16 mm ²
Power Consumption	358 W (201.8 W)	174.64 W (108.47 W)	35.06 W

a): Half of a single TPUv3 chip, b): FP16 w/o sparsity,

c): Sum of the register file, shared memory and L1 cache per SM and the L2 cache size

each head having a dimension of 64, and each state has a dimension of 128. The model’s block size is set to the default value of 256. In terms of accuracy, based on FP16 precision, both PipeFlash and PipeSSD have confirmed that there is no accuracy loss compared to conventional FAs and SSD across eight benchmarks (wikitext-2 [30], Winogrande [46], ARC-challenge and ARC-easy [8], LAMBADA-openai [42], PIQA [6], OpenBookQA [31], and HellaSwag [54]).

Performance. To evaluate the performance of HLX, we developed a custom cycle-level simulator and established three baselines to analyze how performance varies across different hardware platforms as the sequence length changes: the GPUs (NVIDIA A100 80GB [34], H100 80GB [36]), and the TPU (TPUv3 [24]). For the GPU comparison, the baseline was set using results from executing FA-2, FA-3, and SSD operations with GPU-optimized CUDA kernels provided in the GitHub repository [11, 47] for a fair comparison. The performance was analyzed by sweeping the sequence length and the batch size, considering the maximum sequence length and batch size executable on a single GPU. The execution time for each operation and the GPU compute utilization were measured using NVIDIA Nsight Systems [39] and Nsight Compute [38]. For the TPU baseline, a custom cycle-level simulator was developed that emulates a single core (half of a TPUv3 chip), the fundamental unit of a TPUv3, implementing not only the FA-2 and SSD operations (both unfused and fused) but also all operations within the Hybrid model for an end-to-end evaluation. In particular, the MatMul operations were executed with the two 128×128 systolic array-based matrix multiplication units (MXUs), while non-MatMul operations were executed on the vector unit. Due to the DRAM capacity of TPU, an out-of-memory (OOM) error occurs when the sequence length in a single batch exceeds 32K, so the performance analysis was conducted only up to 32K. To ensure a fair comparison with the GPU and TPU baselines, the HLX simulator was implemented in three configurations, as shown in Table 1: HLX³⁰, HLX⁶⁰, and HLX⁶. For comparison with the A100 GPU, the HLX³⁰ configuration uses 30 URSCs, delivering 307.2 TFLOPS with 1935 GB/s of memory bandwidth. Similarly, the HLX⁶⁰ configuration, with 60 URSCs and 2 TB/s of memory bandwidth, is used for comparison with the H100 GPU. The HLX⁶ utilizes 6 URSCs configured to achieve 61.44 TFLOPS and match the TPU’s DRAM bandwidth of 450 GB/s. The

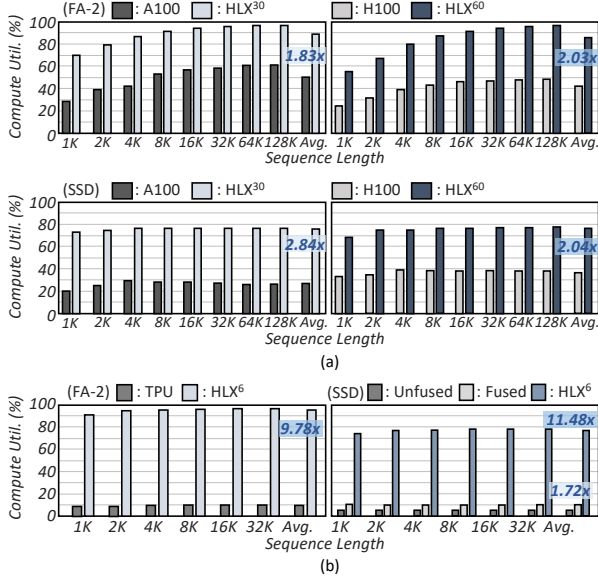


Figure 14: Improvement of compute utilization for FA-2 and SSD over (a) GPU and (b) TPU.

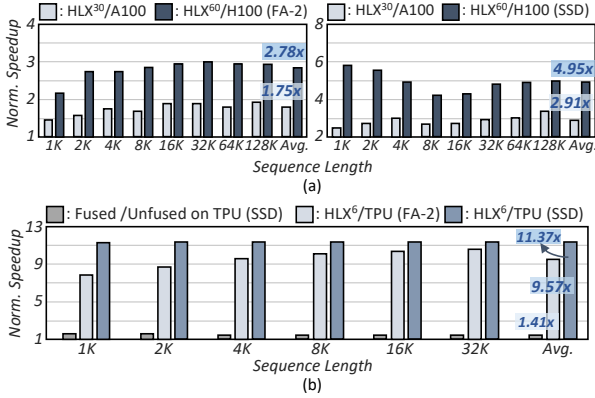


Figure 15: Latency reduction for FA-2 and SSD over (a) GPU and (b) TPU.

DRAM access latency of these configurations was calculated based on their respective DRAM bandwidths.

Area/Power. To estimate the area and power consumption of HLX, a single core of HLX was designed at the RTL level using System Verilog, and the SRAM was compiled. The design was synthesized using Synopsys Design Compiler [48] based on 14nm technology at a frequency of 625MHz. It was also confirmed that the implemented design operates without timing errors at 0.8V and 625MHz, and the synthesized area and power consumption values were compared against the GPU and TPU baselines. In addition, we scaled HLX down to 7nm according to [25] to match the technology process of GPUs. For DRAM power modeling, the HLX simulator incorporated data provided by the DRAM and TPU vendors for the A100 and H100 GPUs using HBM2E and the TPUv3 using HBM2 [23, 32], thereby

enabling accurate power consumption estimation. Meanwhile, the GPU baseline power consumption was measured using NVIDIA-SMI [40].

5.2 Compute Utilization

Comparison to GPU Baseline. Due to the dependency of FA-2 operations on GPUs, compute utilization saturates as the sequence length increases, as shown in Fig. 14(a). HLX alleviates this limitation, achieving approximately 97.5% utilization at 128K, with average improvements of 1.83× and 2.03×, respectively. While FA-2 exhibits increased compute utilization as the sequence length increases, SSD maintains an almost constant compute utilization regardless of sequence length due to its linear characteristics. The H100 achieves a slight increase of compute utilization compared to the A100, but it is still under 40%. In contrast, HLX achieves over 2× higher compute utilization, averaging around 76%, although it does not reach the same high utilization as PipeFlash since d_{head} is half of d_{state} for SSD operations in the given Hybrid model.

Comparison to TPU Baseline. On TPU, compute utilization improves significantly, as shown in Fig. 14(b). This is because, unlike GPU, which are relatively general-purpose, TPU prioritizes compute-intensive dense MatMul operations. In particular, a TPU consists of two MXUs, making it relatively inefficient at handling FA-2 and SSD operations that involve many non-MatMul computations. When running the fused SSD, which improves data reuse compared to the unfused SSD, on TPU, compute utilization increases by 1.72×. However, overall utilization still remains at only about 11%. This indicates that SSD operations are inherently unsuited for MatMul-dominant HW like TPU. In contrast, HLX⁶ achieves an average improvement of 9.78× for FA-2 and 11.48× for SSD in compute utilization compared to the TPU baseline.

5.3 Speedup

FA-2 and SSD. Fig. 15(a) shows the speedups of FA-2 and SSD on HLX compared to the GPU baseline. As the compute utilization of FA-2 improves, HLX achieves an average speedup of 1.75× and 2.78×, respectively. In contrast, even though SSD’s compute utilization remains nearly constant as sequence length increases, the speedup profiles of HLX relative to the A100 and H100 diverge. Against the A100, the speedup of HLX climbs steadily from 1K to 4K because both A100 and HLX gain utilization in this range, yet the incremental benefit is proportionally larger for HLX. Around 8K, the speedup briefly declines. This corresponds to the point where the A100 achieves peak throughput (see Fig. 7(a)), temporarily narrowing the gap. Beyond 8K, the Op/B of A100 falls again, and HLX’s speedup rises once more. The comparison with the H100 shows a different pattern. From a sequence length of 1K to 4K, the speedup of HLX relative to H100 decreases as H100’s throughput increases sharply (see Fig. 7(a)). At 8K, the relative speedup reaches its lowest point. Although both throughput and Op/B on the H100 are lower at 8K than at 4K, this is because the kernels that were unable to fully utilize GPU resources at the short 4K sequence length experience a tiny increase in latency. Beyond 8K, as the Op/B ratio decreases further, the speedup of HLX begins to increase again.

End-to-End Model. Since HLX already has computation units for supporting other computations, such as feed-forward network

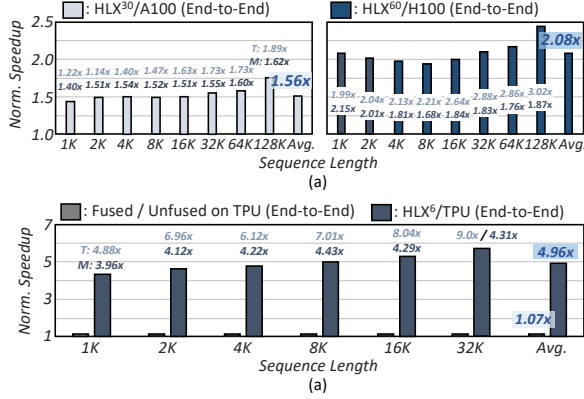


Figure 16: End-to-end speedups over (a) GPU and (b) TPU.

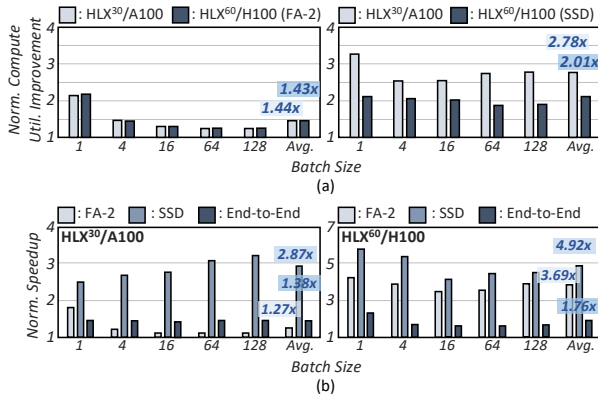


Figure 17: (a) Normalized compute utilization improvement and (b) speedup over GPU with the varying batch size.

(FFN), conv1D, and RMSNorm, we evaluate its end-to-end model latency against the baselines. To ensure a fair comparison focused on pure computational performance, our latency measurements specifically isolate kernel execution times, excluding overheads such as CPU-GPU communication and kernel launching. Figs. 16(a) and (b) illustrate the end-to-end model speedup compared to GPU and TPU baselines. The graphs also show the speedup ratios for a single attention layer and a single Mamba-2 layer (denoted as T and M, respectively) of the Hybrid model. For GPU comparison, since the Hybrid-2.7B model comprises 58 Mamba-2 layers and 6 attention layers, the contribution of the Mamba-2 layers is dominant in the overall speedup. However, as the sequence length increases, the speedup ratio of the attention layers grows more significantly than that of the Mamba-2 layers, leading to a more pronounced overall speedup with longer sequence lengths. Specifically, at a sequence length of 128K, a 1.76 $\times$ (2.45 $\times$) speedup is achieved, with an average speedup of 1.56 $\times$ (2.08 $\times$) compared to the A100 (H100). A similar trend is observed when comparing with TPU, where the speedup of the attention layer increases markedly with sequence length, resulting in an average 4.96 $\times$ speedup.

Table 2: Area and Power Breakdown

HW Components of HLX		
	Area (mm ²)	Power (W)
DPE #0	2.48	2.03
SVPE	1.76	0.85
DPE #1	2.44	2.01
UpE	0.38	0.25
On-chip SRAM	0.68	0.21
Others	0.15	0.05
Total	7.89	5.39

Measured at 625MHz, 0.8V

5.4 Batch Size

To conduct a more comprehensive analysis of HLX, we not only swept through various sequence lengths but also analyzed compute utilization and speedup by increasing the batch size while fixing the sequence length at 1K, as shown in Figs. 17(a) and (b). The results were verified up to a batch size of 128, which is the maximum batch size that can be executed on the GPU baseline.

For compute utilization improvement, FA-2 exhibits a slight decrease as the batch size increases. This is because HLX maintains parallelism along the batch and head dimensions while focusing on resolving dependencies and accelerating computation along the sequence length dimension. Thus, HLX maintains constant compute utilization when the sequence length is fixed. In contrast, the GPU leverages increased parallelism with a larger batch size, leading to higher compute utilization. Nevertheless, HLX achieves an average improvement of 1.44 $\times$. Similarly, for SSD, the compute utilization of HLX remains constant regardless of the batch size, whereas that of A100 increases with the batch size until batch size of 4. However, since the SSD is memory-bound, the compute utilization starts to decline beyond a batch size of 16. Consequently, an average compute utilization improvement of 2.78 $\times$ is achieved. On the other hand, the H100 leverages its higher Op/B and compute utilization compared to the A100, showing a trend where its compute utilization gradually improves up to a batch size of 64 before slightly decreasing at 128. This improvement in compute utilization results in reduced latency for both FA-2 and SSD, yielding an average end-to-end model speedup of from 1.38 $\times$ to 1.76 $\times$ depending on the batch size (see Fig. 17(b)).

5.5 On-Chip SRAM Capacity

As mentioned earlier, PipeFlash and PipeSSD reduce the amount of intermediate data generated during computation by 4.8 $\times$ and 11 $\times$, respectively. This enables HLX to significantly reduce the required on-chip SRAM capacity. Consequently, as shown in Table 1, HLX⁶⁰ and HLX³⁰ require 3.4 $\times$ and 5.55 $\times$ less on-chip SRAM capacity compared to the GPU baseline, and HLX⁶ requires 5.26 $\times$ less compared to the TPU baseline. Additionally, by efficiently achieving speedup with a fixed SRAM capacity irrespective of the sequence length, a smaller area footprint can be maintained.

5.6 Area and Power Breakdown

Table 2 presents the area and power breakdown of HLX. Based on this result, we compare the HLX to GPU and TPU architectures (see Table 1). In terms of area, the HLX⁶⁰ occupies 169mm² (20.8% of the

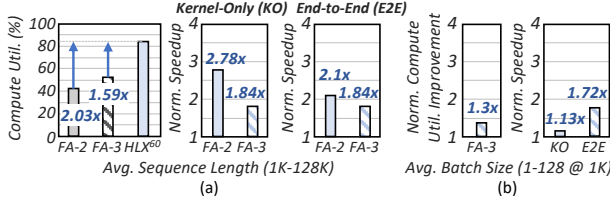


Figure 18: Comparison between PipeFlash on HLX⁶⁰ and FA-3 on H100 GPU according to varying (a) sequence lengths and (b) batch sizes.

H100), the HLX³⁰ is 83.9mm² (10.2% of the A100), and the HLX⁶ is 47.16mm² (14.5% of the TPUv3). Regarding power consumption, the HLX⁶⁰, HLX³⁰, and HLX⁶ consume 42.5%, 63.8%, and 84.4% less power than the H100, A100, and TPUv3, respectively. Within the HLX architecture, the two DPEs are the most dominant components in terms of both area and power consumption, accounting for approximately 62.4% of the total area and 74.9% of the total power usage.

6 Discussion and Related Works

Comparison with FA-3 on H100. Figs. 18(a) and (b) show the comparison between PipeFlash on HLX⁶⁰ and FA-3 on the H100 according to varying sequence lengths and batch sizes. The results indicate that FA-3, being optimized for the Hopper GPU architecture like the H100, achieves improved compute utilization and lower latency compared to FA-2 across all sequence lengths. However, its utilization still saturates at approximately 61%. Consequently, FA-3 on the H100 underperforms relative to the HLX⁶⁰, even though the latter has a lower peak throughput (see Table 1). When sweeping the batch size, the performance gap in compute utilization and kernel latency between the H100 and HLX⁶⁰ narrows, yet the H100 continues to show lower performance. This suggests that while performance has indeed improved over FA-2 due to the H100’s support for asynchronous execution, the GPU cannot fully maximize pipeline parallelism. In contrast, HLX proposes a unified, streamlined architecture based on a fine-grained pipelined dataflow to enable more granular pipeline parallelism. As a result, it remains free from register pressure while achieving high performance in both attention and Mamba-2 operations.

Overhead Analysis of Supporting Both Models. The HLX supports both Transformer and Mamba-2 with minimal HW overhead because our proposed PipeSSD employs a block-level fusion method, similar to that of FA-2, which maximizes HW reuse. This efficiency is further enhanced by the complete sharing of the two DPEs—the primary consumers of chip area and power—by both models. As a result, the HW overhead is modest when compared to accelerators dedicated to a single model. As shown in Table 3, the unified design incurs an area overhead of 3.0% and a power overhead of 2.9% compared to a Transformer-only implementation. This stems from integrating logic for Mamba-2-specific operations, such as conv1D, softplus, and cumsum. Conversely, when compared to a Mamba-2-only design, the overheads are 4.4% in area and 3.5% in power. This increase results from including HW for FA-2’s softmax operation in the RVPE, and adding the reciprocal function and mux/demux

Table 3: HW overhead for supporting both models.

Area Overhead			
Area (mm ²)	HLX	Transformer Only	Mamba-2 Only
DPE #0-1	4.92	4.86	4.92
SVPE	1.76	1.59	1.61
UpE	0.38	0.38	0.20
SRAM & Others	0.83	0.83	0.83
Total	7.89	7.66	7.56
Power Overhead			
Power (W)	HLX	Transformer Only	Mamba-2 Only
DPE #0-1	4.04	4.01	4.04
SVPE	0.85	0.72	0.73
UpE	0.25	0.25	0.18
SRAM & Others	0.26	0.26	0.26
Total	5.39	5.24	5.21

Table 4: Comparison with SOTA accelerators.

Comparison with SOTA Accelerators				
	VGA [25]	MARCA [26]	SOFA [52]	HLX ³⁰
Technology	7 nm	28 nm	28 nm	7 nm
Frequency	1 GHz	1 GHz	1 GHz	625 MHz
Area	52.82 mm ²	221.88 mm ²	5.69 mm ²	83.9 mm ²
Power	41.10 W	10.44 W	3.40 W	108.47 W
On-Chip Mem. Size	-	24 MB	316 KB	15.2 MB
Peak Throughput	49.152 TFLOPS	-	24.423 TOPS	307.2 TFLOPS
Target Computation	FFTConv-based SSM (H3) Only	Mamba-1 Only	Attention Only (w/ Sparsity Handling)	Both Attention and Mamba-2
Fine-grained Pipeline for FA-2 and SSD	X	X	X	O
Speedup over A100 GPU	1.7x (H3-GPT-125M)	1.38x (Mamba-1 2.8B)	-	1.56x (Hybrid-2.7B)

logic to the UpE, both of which are required to support state and O updates.

Comparison with SOTA Accelerators. Table 4 shows a comparison between HLX and state-of-the-art (SOTA) accelerators. First, VGA [25] is a dedicated accelerator for FFT-based convolution, designed to accelerate SSM such as H3 [14]. It operates as a co-processor alongside a GPU or TPU. VGA focuses on offloading memory-intensive FFT-based convolution and state passing to a specialized datapath optimized for generating Vandermonde matrices and utilizing high-bandwidth SRAM. However, the VGA is specifically tailored for earlier SSMs and cannot accommodate FA-2 and newer selective SSMs such as Mamba. MARCA [26] is the first Mamba-1 accelerator specifically tailored for Mamba-1. It introduces a reconfigurable PE array that dynamically performs either linear reductions or element-wise operations, a reusable non-linear function unit that implements exponential and SiLU via approximations, and an operation-wise buffer management strategy to minimize memory traffic. Its reliance on large on-chip memory—occupying 80% of die area—renders it ill-suited for compute-intensive attention workloads. SOFA [52] targets large-scale token-parallel processing in dynamic sparse attention scenarios, addressing the scalability bottlenecks of FA-2. SOFA achieves higher energy and area efficiency by focusing solely on FA-2-based sparse attention, but entirely overlooks the recurrent, memory-intensive Mamba-2 workloads. In contrast, HLX is the first unified accelerator architecture to support both attention and Mamba-2 operations natively. By leveraging a fine-grained pipelined dataflow, it sustains high compute utilization and achieves 1.56× of end-to-end speedup

over an A100 GPU, while MARCA achieves only 1.38 $\times$ of speedup. VGA achieves 1.7 $\times$ of end-to-end speedup, targeting a much smaller model.

Applicability of HLX to Diverse Attention Variants. Recent advances have introduced several attention variants, including group query attention (GQA) [2] and multi-head latent attention (MLA) [12]. GQA reduces the size of KV cache by grouping multiple query heads with the same key and value heads. DeepSeek’s MLA also reduces the KV cache during inference by jointly compressing the key and value into a latent vector with a low-rank projection. More recently, DeepSeek introduced native sparse attention (NSA) [53], which employs a dynamic hierarchical sparse strategy that combines coarse-grained token compression with fine-grained token selection. However, these attention variants do not change the core computations (QK^T , softmax , PV) [47] and support FlashAttention-like block-level fusion [22, 53]. Consequently, PipeFlash can support these diverse attention variants. In other words, despite its simplicity, PipeFlash delivers good applicability across diverse modern attention mechanisms.

Reconfigurable Dataflow Architecture. SambaNova’s recent work [43], SN40L, employs a reconfigurable dataflow architecture with coarse-grained fusion targeting full Transformer decoder layers. high compute throughput via 1,040 Pattern Compute Units (PCUs), achieving up to 638 BF16 TFLOPS. A key aspect of its architecture is its large 520MB on-chip SRAM capacity, which enables aggressive kernel fusion. However, the large on-chip SRAM incurs substantial area and power overhead [7]. In addition, SN40L does not support Mamba-2 or Hybrid models and lacks evaluation for such workloads. In contrast, the proposed HLX architecture adopts fine-grained pipelining and introduces the URSC to support both Transformer and Mamba-2. HLX performs tightly scheduled, pipelined execution with minimal on-chip memory usage (30.4MB for HLX⁶⁰), thereby reducing area and power while maintaining high utilization.

7 Conclusion

Hybrid Transformer-Mamba models combine the high expressiveness of Transformers with the efficiency of Mamba, making them strong candidates to replace traditional Transformer-based models. However, the heterogeneous compute patterns of FA-2 and SSD lead to shifting performance bottlenecks across different workloads, making it challenging to achieve consistent performance. This paper analyzes two core computations, identifying their performance limitations stemming from inter-operation dependencies and low Op/B. We conclude that a fusion-based fine-grained pipelined dataflow is the most effective solution for both operations. Based on this insight, we propose PipeFlash and PipeSSD, and to support these dataflows efficiently, we propose HLX, the first unified accelerator for Hybrid models. HLX features a URSC with fine-grained pipelining and dependency-aware execution, enabling high compute utilization. As a result, compared to the A100, HLX³⁰ delivers an average 1.75 $\times$, 2.91 $\times$ and 1.56 $\times$ speedups for FA-2, SSD, and end-to-end latency across variable sequence lengths. It also boosts batching performance by 1.38 $\times$ while consuming 89.8% less area and 63.8% less power. Also, HLX⁶⁰ achieves an average 2.78 $\times$, 1.84 $\times$, and 4.95 $\times$ speedups for FA-2, FA-3, and SSD over H100. For end-to-end and

batching performance, HLX⁶⁰ achieves a 2.08 $\times$ and 1.76 $\times$ (1.84 $\times$ and 1.72 $\times$) speedup when running FA-2 (FA-3) with 79.2% less area and 42.5% less power consumption. Overall, these results demonstrate HLX’s effectiveness as an efficient accelerator for Hybrid models.

Acknowledgments

This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.IITP-2025-RS-2022-II221037, Development of High Performance Processing In Memory Technology based on DRAM, IITP-2025-RS-2023-00256472, Graduate School of Artificial Intelligence Semiconductor, and IITP-2025-RS-2020-II201847, Information Technology Research Center (ITRC)).

References

- [1] Shantanu Acharya, Fei Jia, and Boris Ginsburg. 2024. Star attention: Efficient llm inference over long sequences. *arXiv preprint arXiv:2411.17116* (2024).
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245* [cs.CL] <https://arxiv.org/abs/2305.13245>
- [3] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>.
- [4] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalisina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. 2024. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2402.18668* (2024).
- [5] Michael Bauer, Henry Cook, and Bruce Khailany. 2011. CudaDMA: Optimizing GPU memory bandwidth via warp specialization. In *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11. <https://doi.org/10.1145/2063384.2063400>
- [6] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2019. PIQA: Reasoning about Physical Commonsense in Natural Language. *arXiv:1911.11641* [cs.CL] <https://arxiv.org/abs/1911.11641>
- [7] Keonhee Cho, Heekyung Choi, In Jun Jung, Jisang Oh, Tae Woo Oh, Kiryong Kim, Giseok Kim, Taemin Choi, Changsu Sim, Taejoong Song, and Seong-Ook Jung. 2022. SRAM Write- and Performance-Assist Cells for Reducing Interconnect Resistance Effects Increased With Technology Scaling. *IEEE Journal of Solid-State Circuits* 57, 4 (2022), 1039–1048. <https://doi.org/10.1109/JSSC.2021.3138785>
- [8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [9] Neal C. Crago, Sana Damani, Karthikeyan Sankaralingam, and Stephen W. Keckler. 2024. WASP: Exploiting GPU Pipeline Parallelism with Hardware-Accelerated Automatic Warp Specialization. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1–16. <https://doi.org/10.1109/HPCA57654.2024.00086>
- [10] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [11] Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).
- [12] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyu Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu,

- Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yuduan Wang, Yuheng Zou, Yukun Xia, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434* [cs.CL] <https://arxiv.org/abs/2405.04434>
- [13] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks* 107 (2018), 3–11.
- [14] Daniel Y Fu, Tri Dao, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. 2022. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052* (2022).
- [15] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. 2024. The Zamba2 Suite: Technical Report. *arXiv preprint arXiv:2411.15242* (2024).
- [16] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. 2024. Zamba: A compact 7b smm hybrid model. *arXiv preprint arXiv:2405.16712* (2024).
- [17] Google. 2025. Gemini 2.0 is now available to everyone. <https://blog.google/technology/google-deepmind/gemini-model-updates-february-2025>.
- [18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kunal Lakhotia, Lauren Rantala-Young, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimopoulaki, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Celebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stéphane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpeierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bhamambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Han-nah Wang, Hanwen Zha, Haroun Hafeez, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madihan Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanachandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangrabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuangqing Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiao Cheng Tang, Xiaojuan Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
- [19] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [20] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825* [cs.CL] <https://arxiv.org/abs/2310.06825>
- [21] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2024. Mistral of Experts. *arXiv:2401.04088* [cs.LG] <https://arxiv.org/abs/2401.04088>
- [22] Shengyu Liu Jiaoshi Li. 2025. FlashMLA: Efficient MLA decoding kernels. <https://github.com/deepseek-ai/FlashMLA>.
- [23] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4: Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on*

- Computer Architecture (ISCA). 1–14. <https://doi.org/10.1109/ISCA52012.2021.00010>
- [24] Norman P Joppa, Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. A domain-specific supercomputer for training deep neural networks. *Commun. ACM* 63, 7 (2020), 67–78.
- [25] Seung Yul Lee, Hyunseung Lee, Jihoon Hong, SangLyul Cho, and Jae W Lee. 2024. VGA: Hardware Accelerator for Scalable Long Sequence Model Inference. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1444–1457.
- [26] Jinhao Li, Shan Huang, Jiaming Xu, Jun Liu, Li Ding, Ningyi Xu, and Guohao Dai. 2025. MARCA: Mamba Accelerator with Reconfigurable Architecture. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (Newark Liberty International Airport Marriott, New York, NY, USA) (ICCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 234, 9 pages. <https://doi.org/10.1145/3676536.3676798>
- [27] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirum, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. *arXiv:2403.19887 [cs.CL]* <https://arxiv.org/abs/2403.19887>
- [28] Hanxiao Liu, Zihang Dai, David So, and Quoc V Le. 2021. Pay attention to mlp. *Advances in neural information processing systems* 34 (2021), 9204–9215.
- [29] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. 2024. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE Computer Society, Los Alamitos, CA, USA, 656–667. <https://doi.org/10.1109/IPDPS57955.2024.00064>
- [30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [31] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789* (2018).
- [32] Ki-Il Moon, Ho-Young Son, and Kangwook Lee. 2023. Advanced Packaging Technologies in Memory Applications for Future Generative AI Era. In *2023 International Electron Devices Meeting (IEDM)*. IEEE, 1–4.
- [33] Nandeeka Nayak, Xinrui Wu, Toluwanimi O Odemuyiwa, Michael Pellauer, Joel S Emer, and Christopher W Fletcher. 2024. Fusemax: Leveraging extended einsums to optimize attention accelerator design. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1458–1473.
- [34] NVIDIA. 2025. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/>.
- [35] NVIDIA. 2025. NVIDIA A100 Tensor Core GPU Architecture. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>.
- [36] NVIDIA. 2025. NVIDIA H100 Tensor Core GPU Architecture. <https://www.nvidia.com/en-us/data-center/h100/>.
- [37] NVIDIA. 2025. NVIDIA H100 Tensor Core GPU Architecture. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>.
- [38] NVIDIA. 2025. NVIDIA Nsight Compute. <https://developer.nvidia.com/nsight-compute>.
- [39] NVIDIA. 2025. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>.
- [40] NVIDIA. 2025. NVIDIA System Management Interface SMI. <https://developer.nvidia.com/system-management-interface>.
- [41] OpenAI. 2024. OpenAI Documentation - Models. <https://platform.openai.com/docs/models/overview>.
- [42] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031* (2016).
- [43] Raghu Prabhakar, Ram Sivaramakrishnan, Darshan Gandhi, Yun Du, Mingran Wang, Xiangyu Song, Kejie Zhang, Tianren Gao, Angela Wang, Xiaoyan Li, Yongning Sheng, Joshua Brot, Denis Sokolov, Apurv Vivek, Calvin Leung, Arjun Sabnis, Jiayu Bai, Tuowen Zhao, Mark Gottscho, David Jackson, Mark Luttrell, Manish K. Shah, Zhengyu Chen, Kaizhao Liang, Swayambhoo Jain, Urmish Thakker, Dawei Huang, Sumti Jairath, Kevin J. Brown, and Kunle Olukotun. 2024. SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1353–1366. <https://doi.org/10.1109/MICRO61859.2024.00100>
- [44] PyTorch. 2025. PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>.
- [45] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. 2024. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522* (2024).
- [46] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
- [47] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2025. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2025), 68658–68685.
- [48] Synopsys. 2025. Synopsys Design Compiler. <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>.
- [49] Jamba Team, Barak Lenz, Alan Araz, Amir Bergman, Avshalom Manevich, Barak Peleg, Ben Aviram, Chen Almagor, Clara Fridman, Dan Padnos, Daniel Gissin, Daniel Jannai, Dor Muhlgay, Dor Zimberg, Edden M Gerber, Elad Dolev, Eran Krakovsky, Erez Safahi, Erez Schwartz, Gal Cohen, Gal Shachaf, Haim Rozenblum, Hofit Bata, Ido Blass, Inbal Magar, Itay Dalmedigos, Jhonathan Osin, Julie Fadlon, Maria Rozman, Matan Danos, Michael Gokhman, Mor Zusman, Naama Gidron, Nir Ratner, Noam Gat, Noam Rozen, Oded Fried, Ohad Leshno, Omer Antverg, Omri Abend, Opher Lieber, Or Dagan, Orit Cohavi, Raz Alon, Ro'i Belson, Roi Cohen, Rom Gilad, Roman Glozman, Shahar Lev, Shaked Meirum, Tal Delbari, Tal Ness, Tomer Asida, Tom Ben Gal, Tom Braude, Uriya Pumerantz, Yehoshua Cohen, Yonatan Belinkov, Yuval Globerson, Yuval Peleg Levy, and Yoav Shoham. 2024. Jamba-1.5: Hybrid Transformer-Mamba Models at Scale. *arXiv:2408.12570 [cs.CL]* <https://arxiv.org/abs/2408.12570>
- [50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [51] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, Garvit Kulshreshtha, Vartika Singh, Jared Casper, Jan Kautz, Mohammad Shoeybi, and Bryan Catanzaro. 2024. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887* (2024).
- [52] Huizheng Wang, Jiahao Fang, Xinru Tang, Zhiheng Yue, Jinxi Li, Yubin Qin, Sihan Guan, Qizhe Yang, Yang Wang, Chao Li, Yang Hu, and Shouyi Yin. 2024. SOFA: A Compute-Memory Optimized Sparsity Accelerator via Cross-Stage Coordinated Tiling. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1247–1263. <https://doi.org/10.1109/MICRO61859.2024.00093>
- [53] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. *arXiv:2502.11089 [cs.CL]* <https://arxiv.org/abs/2502.11089>
- [54] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).